RocketBase
Fullstack-Developer-Team

Für Beispielbetrieb GmbH (fiktiv)

Sicherheitsdurchsicht Auftragsportal

nach der Systematik des OWASP Application Security Verification Standard

Auftraggeber Beispielbetrieb GmbH	Prüfgegenstand Auftragsportal mit Terminbuchung und Online-Zahlung, TypeScript/Node, PostgreSQL 16, Anbindung an Zahlungsanbieter, Warenwirtschaft und Mailversand	Prüfstand Repository vom 12.10., Migrationen bis 064
Prüfzeitraum 13. bis 16.10.	Prüfer Marten Prieß, RocketBase	

Dies ist ein Musterbericht. Betrieb, System und Befunde sind ausgedacht und keinem Kunden entnommen. Aufbau, Prüfverfahren, Maßstab und Schreibweise entsprechen unseren echten Berichten. Ein echter Bericht hat je nach Umfang 10 bis 15 Seiten.

0. Einordnung und Grenzen

Diese Durchsicht ist ein **Review, kein Penetrationstest und kein Testat.** Sie folgt der Kapitelsystematik des OWASP ASVS, damit nachvollziehbar bleibt, was geprüft wurde, und beansprucht keine Zertifizierung nach einer ASVS-Stufe. Sie geht an mehreren Stellen über den ASVS hinaus: Geschäftslogik unter gleichzeitigen Zugriffen, Verhalten bei Ausfall von Gegenstellen, Auslieferung und Betrieb sowie der Abgleich der Dokumentation mit dem Code.

Verfahren. Statische Durchsicht von Quellcode, Migrationen, Infrastrukturvorlagen und Pipelines. Dazu eine lokal aufgesetzte Instanz mit allen Migrationen, gegen die das Verhalten bei gleichzeitigen Anfragen und bei nicht erreichbaren oder langsamen Gegenstellen gemessen wurde. Zahlungsanbieter und Warenwirtschaft wurden dafür nachgebaut. Abhängigkeiten mit `npm audit`, einem OSV-Abgleich und Trivy geprüft. Gelesen wurde zweimal: einmal mit README und Projektdokumentation, einmal nur entlang des Codes, Kommentare darin als Behauptung behandelt. Zum Schluss wurde jeder Befund noch einmal von der Fundstelle aus nachgeprüft; was dabei nicht hielt, ist gestrichen oder herabgestuft. Abgeglichen wurde gegen die eigene Liste offener Punkte; `neu` heißt: dort nicht geführt.

Prüfgegenstand im Zusammenhang. Kunden buchen im Portal einen Termin und bezahlen online. Nach der Zahlung geht der Auftrag an die Warenwirtschaft, der Kunde erhält eine Bestätigung per Mail. Erstattungen erzeugt das Portal als Bankdatei. Geprüft wurde, ob es diese Aufgaben auch dann sauber erledigt, wenn zwei Anfragen gleichzeitig kommen oder eine Gegenstelle ausfällt.

Nicht geprüft: die Warenwirtschaft selbst, die reale Konfiguration beim Cloud-Anbieter (Rechte, Netz, Sicherungen), produktive Daten, kein aktiver Angriff, keine Last. Wo ein Befund von einer dieser Stellen abhängt, steht das am Befund.

Maßstab. Ein Befund wiegt schwer, wenn er im Betrieb unbemerkt bleibt, viele Kunden gleichzeitig trifft, Geld oder Daten dauerhaft beschädigt oder sich nach dem Go-live nur noch teuer beheben lässt. Befunde niedriger Wirkung sind nicht aufgeführt.

Diese Seite gibt es nur im Muster: welche Felder ein Software-Check abdeckt und wie Befunde eingestuft werden.

Was geprüft wird

PRÜFFELD	TYPISCHE FRAGEN	GRUNDLAGE
Anmeldung, Sitzung, Rechte	Ist jeder Zugang standardmäßig gesperrt? Kann Kunde A über eine geänderte Nummer die Daten von Kunde B sehen? Kommen Rollen aus der Datenbank oder nur aus dem Token?	OWASP ASVS V2–V4
Eingaben, Schnittstellen, Geheimnisse	Sind alle Abfragen parametrisiert? Liegen Schlüssel im Code oder im Tresor? Wer kann mit einem Schnittstellenschlüssel was abrufen, und läuft er ab?	OWASP ASVS V5, V6, V13
Geschäftslogik und gleichzeitige Zugriffe	Was passiert, wenn zwei Leute gleichzeitig auf den letzten Platz buchen? Kann eine Zahlung zweimal hinausgehen? Gibt es Grenzwerte in der Datenbank oder nur im Code?	OWASP ASVS V11, dazu Messung an laufender Instanz
Ausfall von Gegenstellen	Was tut das System, wenn Zahlungsanbieter, Warenwirtschaft oder Mailversand nicht oder langsam antworten? Wird eine verlorene Nachricht automatisch nachgeholt?	über den ASVS hinaus, gemessen mit nachgebauter Gegenstelle
Datenmodell	Fehlen Indizes auf Fremdschlüsseln? Lassen sich Nachweise ändern, die unveränderlich sein müssten? Laufen alle Migrationen auf einer leeren Datenbank?	über den ASVS hinaus
Auslieferung und Betrieb	Prüft ein zweiter Mensch, bevor etwas live geht, oder setzt die KI die Freigabe selbst? Meldet die Zustandsprüfung einen echten Ausfall? Gibt es einen Rückweg?	OWASP ASVS V1, V7, V14 und darüber hinaus
Datenschutz und Löschung	Gibt es ein Löschkonzept? Was steht in den Logs? Wohin gehen Daten an Dritte, etwa an ein Sprachmodell?	OWASP ASVS V8, DSGVO
Abhängigkeiten und Versionen	Bekannte Schwachstellen in Bibliotheken? Laufzeitumgebung noch mit Sicherheitspatches?	OWASP ASVS V10, V14
Dokumentation gegen Code	Stimmt, was im Runbook und in der Übergabe steht, mit dem Code überein? Wer übernimmt, liest zuerst die Dokumentation.	über den ASVS hinaus

Wie eingestuft wird

STUFE	BEDEUTUNG	WANN ZU BEHEBEN
kritisch	Trifft viele Kunden gleichzeitig oder beschädigt Geld oder Daten dauerhaft, oft ohne dass es jemand merkt.	sofort, vor dem Go-live
hoch	Deutlicher Schaden im Einzelfall, oder nach dem Go-live nur noch teuer zu beheben.	vor dem Go-live
mittel	Begrenzte Wirkung, aber auf Dauer nicht tragbar.	im laufenden Betrieb

Zu jedem Befund steht, ob er in euren Unterlagen schon geführt ist oder neu, ob er gemessen oder nur gelesen wurde, und welche Empfehlung gilt. Was ausdrücklich so bleiben darf, steht in einem eigenen Kapitel.

Was ihr bekommt

Management-Summary	Eine Seite für die Geschäftsführung, ohne Fachbegriffe. Muster auf der nächsten Seite.
Sicherheitsdurchsicht	Der Bericht mit allen Befunden, Fundstellen und Empfehlungen. Auszug ab Seite 4.
Vorab-Stichpunkte	Die wichtigsten Befunde kurz vor dem Bericht, damit das Team schon anfangen kann.
Abschlusstermin	Etwa anderthalb Stunden, die wichtigsten Befunde live vorgeführt, Reihenfolge gemeinsam festgelegt.
Papier zum Betrieb	Themen, die man im Code nicht sieht. Bei Bedarf Fragen an eure Dienstleister. Auszüge im Anhang.

RocketBase
Fullstack-Developer-Team

Für Beispielbetrieb GmbH (fiktiv)

Auftragsportal: Ergebnis des Software-Checks

Management-Summary

Für	Prüfstand	Prüfer
Geschäftsführung	Codestand vom 12.10.	Marten Prieß, RocketBase

Das Auftragsportal ist sicher gebaut, aber noch nicht betriebsfest. Ein Angreifer von außen kommt nach meiner Prüfung nicht an Kundendaten. Anmeldung, Zugangskontrolle und Umgang mit Zugangsdaten entsprechen dem Stand der Technik. Die Risiken liegen dort, wo das Portal auf Zahlungsanbieter und Warenwirtschaft trifft, bei gleichzeitigen Buchungen und im Weg, auf dem Änderungen live gehen.

Die Risiken

Bezahlte Buchungen werden storniert. Ist der Zahlungsanbieter eine Stunde lang nicht erreichbar, storniert das Portal jede Buchung, deren Zahlung in dieser Zeit bestätigt werden sollte. Der Kunde hat bezahlt, der Termin ist weg.

Ein Termin kann zweimal vergeben werden, eine Erstattung zweimal hinausgehen. Kommen zwei Anfragen gleichzeitig, nimmt das Portal beide an. Gemessen, nicht vermutet.

Ein einzelner Schlüssel öffnet alle Kundendaten. Der Zugang der Warenwirtschaft läuft nie ab und liefert die Daten aller Kunden, nicht nur der offenen Aufträge.

Änderungen gehen ohne zweiten Menschen live. Der KI-Assistent setzt die Freigabe selbst, und ein Ausfall nach einer Auslieferung fällt nicht auf, weil die Zustandsmeldung die Datenbank nicht prüft.

Vor dem Go-live

Stornierung nur bei echter Ablehnung durch den Zahlungsanbieter, Sperren bei Buchung und Erstattung, ein Pflicht-Review durch eine Person vor jeder Auslieferung, eine Zustandsmeldung, die einen Ausfall erkennt. Zusammen rund drei Entwicklertage.

Zur Entscheidung: Erstattungen über 500 € gehen ohne zweite Freigabe in die Bankdatei. Das ist so gewollt, sollte aber bewusst entschieden sein.

Was in den IT-Betrieb übergehen sollte

Freigabe von Auslieferungen. Verwaltung der Schnittstellenschlüssel mit Ablaufdatum. Alarmierung bei Ausfall mit klarer Zuständigkeit. Eine erprobte Rücksicherung der Datenbank.

Aufwand: Alle Befunde der Stufen kritisch und hoch etwa eine Entwicklerwoche. Danach im laufenden Betrieb die mittleren Befunde und die Dokumentation auf den Stand des Codes.

1. Ergebnis in Kürze

Das Portal ist an den klassischen Stellen sorgfältig gebaut. Anmeldung, Zugangskontrolle, Umgang mit Geheimnissen und Schutz gegen Einschleusung sind ohne Befund. **Eine Lücke, über die ein anonymer Angreifer aus dem Internet an Kundendaten kommt, wurde nicht gefunden.**

Die Befunde liegen woanders. Das Portal verlässt sich darauf, dass jede Gegenstelle antwortet und nie zwei Anfragen zur selben Zeit dieselben Daten anfassen.

Ausfall der Gegenstellen. Jede Antwort des Zahlungsanbieters außer einer Bestätigung zählt als Ablehnung, auch eine Zeitüberschreitung. Nach drei solchen Antworten wird die Buchung storniert (3.1.1).

Gleichzeitige Zugriffe. Buchung und Erstattung lesen, entscheiden im Speicher und schreiben ohne Sperre (4.1.1, 4.1.2).

Auslieferung. Änderungen werden ohne menschliche Prüfung zusammengeführt und ausgeliefert, die Zustandsmeldung erkennt keinen Ausfall (6.1.1, 6.1.2).

kritisch	hoch	mittel
1	5	12

Von den sechs Befunden der Stufen kritisch und hoch ist einer als bewusste Entscheidung im Code festgehalten, einer in euren Unterlagen geführt, vier sind neu. Dazu kommen vier Stellen, an denen die Dokumentation etwas anderes sagt als der Code (Kapitel 8).

2. Was hält

✓ **Anmeldung vollständig.** Signatur, Aussteller, Zielgruppe und Algorithmus festgelegt; ohne Konfiguration startet die Anwendung nicht (`auth/verify.ts:12-58`).

✓ **Zugänge standardmäßig zu.** Jede Route ohne ausdrückliche Freigabe verlangt eine Anmeldung. Gemessen: alle 46 geschützten Routen ohne Nachweis abgelehnt. Ein Zugriff von Kunde A auf Buchungen von Kunde B wurde nicht gefunden.

✓ **Geheimnisse und Einschleusung.** Nichts im Quellcode, in Tests oder im Frontend-Bündel. Alle Abfragen parametrisiert.

✓ **Geldbeträge in ganzen Cent,** ohne Fließkomma, durchgängig vom Formular bis zur Bankdatei.

✓ **Übergabe an die Warenwirtschaft** mit Zustandsautomat und eingefrorenem Beleg. Eine Stelle, die als Vorlage für die eigenen Reparaturen taugt (`erp/uebergabe.ts:40-96`).

✓ **Entscheidungen nummeriert,** mit Datum und Begründung. Das macht ein Review in wenigen Tagen überhaupt erst möglich.

3. Ausfallsicherheit gegenüber Zahlungsanbieter, Warenwirtschaft und Mailversand

3.1 Vor dem Go-live

3.1.1 kritisch Ein Ausfall beim Zahlungsanbieter storniert bezahlte Buchungen — V11

`payment/bestaetigung.ts:88-131` zählt jedes Ergebnis außer einer Bestätigung als Fehlversuch: Zeitüberschreitung, 503 und 429 wiegen so schwer wie eine echte Ablehnung. Nach drei Fehlversuchen im 20-Minuten-Takt storniert `buchung/stornieren.ts` die Buchung und gibt den Termin frei. **Gemessen** mit einer nachgebauten Gegenstelle, die 503 antwortet: Nach 61 Minuten sind alle offenen Buchungen storniert, die Zahlung beim Anbieter bleibt bestehen. Der Kunde bekommt eine Stornomail für einen bezahlten Termin. **Empfehlung:** Nur eine ausdrückliche Ablehnung zählt; Transportfehler, 5xx und 429 setzen den letzten Fehler und lösen einen Betriebsalarm aus. Bekannt als Entscheidung im Kommentar; die Folge bei längerem Ausfall ist dort nicht betrachtet.

3.2 Weitere Befunde mittel

> **Die Übergabe an die Warenwirtschaft hat keine Wiederholung.** Scheitert der Aufruf, steht der Auftrag auf `uebergabe_offen`, und nichts holt ihn automatisch nach. neu

> **Der Mailversand hat kein Zeitlimit** und läuft in derselben Anfrage wie die Buchung. Ein langsamer Mailanbieter hält die Buchung offen.

4. Buchungen, Zahlungen und gleichzeitige Zugriffe

4.1 Vor dem Go-live

4.1.1 hoch Zwei Buchungen auf den letzten freien Termin werden beide angenommen — V11

`buchung/anlegen.ts:40-72` liest den freien Platz, entscheidet im Speicher und schreibt danach, ohne Sperre. **Gemessen** mit einem Integrationstest gegen eine echte Datenbank: erwartet 1 angenommen und 1 abgelehnt, tatsächlich 2 angenommen. Die nächtliche Bereinigung löscht die doppelte Buchung, ohne einen der Kunden zu informieren. **Empfehlung:** Zeilensperre beim Lesen, Schreiben in derselben Transaktion; Vorbild ist `lager/reservierung.ts:18-35`, die es bereits richtig macht. Der Test liegt dem Bericht bei. neu

4.1.2 hoch Eine Erstattung kann zweimal in die Bankdatei gelangen — V11

`erstattung/zahllauf.ts:157` liest freigegebene Erstattungen ohne Sperre und markiert sie erst nach dem Hochladen. Starten zwei Zahlläufe kurz nacheinander, etwa durch einen Doppelklick, enthalten beide Dateien dieselben Erstattungen. **Empfehlung:** Sperre im Zahllauf, Festschreibung vor dem Hochladen, Grenzwert `CHECK (betrag > 0)` in der Datenbank. In der Liste offener Punkte teilweise geführt.

4.2 Weitere Befunde mittel

> **Keine Grenzwerte in der Datenbank.** Null CHECK-Bedingungen in 64 Migrationen; Betrag und Menge sind nur im Code geprüft.

> **Fremdschlüssel ohne Index.** 31 von 38, darunter alle auf `buchungen`. Mit wachsenden Daten werden Löschen und Auswertungen langsam.

5. Zugangsdaten und Rechte

5.1 Vor dem Go-live

5.1.1 hoch Ein Schlüssel der Warenwirtschaft liefert die Daten aller Kunden — V4

`api/erp-export.ts:22-61` gibt mit einem einzigen, nie ablaufenden Schlüssel Name, Adresse und Buchungen aller Kunden heraus, nicht nur der offenen Aufträge. Der Schlüssel liegt zusätzlich in der Konfiguration des Dienstleisters der Warenwirtschaft. **Empfehlung:** eigener, wechselbarer Schlüssel je Gegenstelle mit Ablaufdatum; Ausgabe nur der Aufträge, die übergeben werden sollen. neu

5.2 Weitere Befunde mittel

> **Die Rolle kommt aus dem Token, nicht aus der Datenbank.** Wer gesperrt wird, behält seine Rechte bis zum Ablauf, bis zu 12 Stunden.

> **Die Begrenzung der Anfragen ist installiert, aber nirgends eingebunden.** Anmeldung und Kontaktformular lassen sich beliebig oft aufrufen. neu

6. Auslieferung und Betrieb

6.1 Vor dem Go-live

6.1.1 hoch Änderungen gehen ohne zweiten Menschen live — V1

`.github/workflows/automerge.yml` führt Änderungen zusammen, sobald ein Label gesetzt ist. Dieses Label setzt der KI-Assistent selbst. Ausliefern lässt sich jeder Zweig, nicht nur der Hauptzweig. **Empfehlung:** Pflicht-Review durch eine Person auf dem Hauptzweig, Auslieferung nur von dort mit grünem Testlauf. neu

6.1.2 hoch Die Zustandsmeldung sagt „ok“, auch wenn die Datenbank weg ist — V7

`api/health.ts` prüft nur, ob der Server antwortet. Genau diese Meldung entscheidet nach jeder Auslieferung, ob die neue Version live bleibt. **Gemessen:** Datenbank gestoppt, Antwort `200 {"status":"ok"}`. Eine Alarmierung gibt es nicht, ein Ausfall fällt erst auf, wenn ein Kunde anruft. **Empfehlung:** Abfrage gegen die Datenbank mit kurzem Zeitlimit, bei Fehler 503; Alarm mit fester Zuständigkeit; Auslieferung mit Rückweg.

6.2 Weitere Befunde mittel

> **214 Integrationstests laufen in der Pipeline nie,** weil dort keine Datenbank bereitsteht; sie werden still übersprungen.

> **Die Laufzeitumgebung bekommt seit April keine Sicherheitspatches mehr.** Das meldet kein Werkzeug, man muss die Support-Zyklen kennen.

7. Datenschutz und Löschung

7.1 Weitere Befunde mittel

> **Es gibt kein Löschkonzept.** Buchungen und Kundendaten bleiben unbegrenzt, auch nach Ablauf der Aufbewahrungsfrist.

> **Einwilligungen lassen sich ändern und löschen.** Gerade sie müssten als Nachweis unveränderlich sein.

> **Die Schwärzung im Logging zielt auf Feldnamen, die der Code nicht verwendet.** Der Test dazu ist grün, weil er genau diese Schreibweise prüft.

8. Dokumentation gegen Code

Die Dokumentation ist umfangreich, und gerade deshalb wird sie gelesen werden. An diesen Stellen sagt sie etwas anderes:

DOKUMENT	AUSSAGE	CODE
`docs/runbook.md:41`	Der Nachtlauf holt gescheiterte Übergaben nach	Er liest nur neue Aufträge (3.2)
`docs/betrieb.md:12`	Tägliche Sicherung mit Test der Rücksicherung	Sicherung an, eine Rücksicherung ist nirgends beschrieben; ohne Cloud-Zugang nicht prüfbar
`docs/sicherheit.md:30`	Anfragen werden begrenzt	Paket installiert, nicht eingebunden (5.2)
`README.md:8`	Tests laufen bei jeder Änderung	Integrationstests werden übersprungen (6.2)

9. Empfohlene Reihenfolge

Sofort, vor dem Go-live.

1. Fehlversuche beim Zahlungsanbieter nach Art unterscheiden, Betriebsalarm statt Stornierung (3.1.1)
2. Sperren bei Buchung und Zahllauf, Festschreibung vor dem Hochladen, Grenzwerte in der Datenbank (4.1.1, 4.1.2)
3. Pflicht-Review durch eine Person, Auslieferung nur vom Hauptzweig (6.1.1)

Kurzfristig, geringer Aufwand. Zusammen wenige Tage.

4. Zustandsmeldung mit Datenbankprüfung, Alarmierung, Auslieferung mit Rückweg (6.1.2)
5. Eigener Schlüssel für die Warenwirtschaft mit Ablaufdatum, Ausgabe nur offener Aufträge (5.1.1)
6. Rolle aus der Datenbank, Begrenzung der Anfragen einbinden (5.2)

Mittelfristig, größerer Umfang.

7. Nachholen gescheiterter Übergaben, Zeitlimits für alle Gegenstellen (3.2)
8. Integrationstests mit Datenbank in der Pipeline, Indizes auf Fremdschlüsseln (4.2, 6.2)
9. Löschkonzept, unveränderliche Einwilligungen, Schwärzung im Logging (Kapitel 7)
10. Dokumentation auf den Stand des Codes bringen (Kapitel 8)

Zur Entscheidung beim Beispielbetrieb, kein Befund. Erstattungen gehen ohne Betragsgrenze und ohne zweite Freigabe in die Bankdatei. Das ist offen benannt. Eine zweite Freigabe ab einem festen Betrag wäre die vorsichtigere Wahl.

10. Schlussbemerkung

Das Auftragsportal ist kein Sanierungsfall. Die Kapitel, in denen Prüfungen dieser Art sonst die schwersten Mängel finden, sind hier ohne Befund. Die Befunde liegen dort, wo das Portal auf andere Systeme trifft und wo zwei Dinge gleichzeitig passieren. Mit wenigen Testkunden fällt das nicht auf. Im Betrieb sieht es niemand mehr. Die Punkte unter „sofort“ sind einzeln klein und schützen am meisten.

Anhang A · Papier zum Betrieb, Auszug

Themen, die man im Repository nicht sieht, sondern erst im Betrieb. Sie entstehen meist im Abschlusstermin und werden danach eigens aufgeschrieben.

Der Mailweg

Eine unzustellbare Mail wird zur Kundennachricht. Existiert eine Adresse nicht, schickt der Mailserver der Gegenseite eine Unzustellbarkeitsmeldung mit der Buchungsnummer im Text. Das Portal hält sie für eine Kundenantwort und hängt sie an die Buchung. Der Vorgang sieht bearbeitet aus. Er ist es nicht.

Es gibt kein Feld für den Zustellstatus. Ob eine Bestätigung angekommen ist, weiß niemand.

Die Absenderdomain ist nicht vollständig eingerichtet. Große Anbieter sortieren solche Mails zunehmend aus. Niemand merkt, wenn ab morgen jede zehnte Mail zurückkommt.

Das Backup

Es gibt eins. Die Frage ist, ob schon einmal jemand zurückgespielt hat. Ein vollständiger Rücksprung löscht alles, was seither gebucht wurde. Der übliche Weg ist ein zweites Exemplar der Datenbank, aus dem man gezielt holt, was fehlt. Das muss jemand dürfen und einmal geübt haben.

Ein Protokoll der Vorgänge

Das Portal hält fest, was sich an einer Buchung geändert hat, aber nicht, was gescheitert ist. Bei einer Rückfrage „Warum habe ich keine Bestätigung bekommen?“ gibt es keine Antwort. Ein Eintrag je Schritt, auch für Fehlschläge, beantwortet solche Fragen in Sekunden.

Was sonst noch aussteht

> Wer bekommt nachts eine Nachricht, wenn das Portal steht, und was tut er dann?

> Ein Lasttest vor der ersten großen Aktion, etwa einem Newsletter mit Terminangebot.

> Der Weg von der Testumgebung in die Produktion und zurück, aufgeschrieben und einmal geübt.

Anhang B · Fragen an euren Dienstleister, Auszug

Manches hängt an Systemen, die nicht im geprüften Code liegen. Dafür gibt es eine eigene Liste, die ihr direkt weitergeben könnt. Zu jeder Frage steht, bei wem die Antwort vermutlich liegt und warum ich frage.

1. Was passiert, wenn dieselbe Buchung zweimal bei der Warenwirtschaft ankommt? *eher Dienstleister*

Warum ich frage: Das Portal wiederholt bei einer Zeitüberschreitung. Erkennt die Warenwirtschaft die Wiederholung nicht, entsteht ein zweiter Auftrag.

2. Wo liegt der Schlüssel für die Schnittstelle, und ließe er sich ohne Ausfall wechseln? *eher Dienstleister*

Warum ich frage: Er öffnet heute die Daten aller Kunden (5.1.1). Ein Wechsel muss geplant werden, bevor er nötig ist.

3. Wie lange speichert die Warenwirtschaft die übergebenen Kundendaten? *vermutlich ihr*

Warum ich frage: Für ein Löschkonzept zählt jede Kopie, nicht nur die im Portal.

4. Wer bekommt eine Nachricht, wenn der Zahlungsanbieter Störungen meldet? *vermutlich ihr*

Warum ich frage: Der Anbieter veröffentlicht Störungen auf einer Statusseite. Wer sie abonniert, weiß vor dem ersten Kundenanruf Bescheid.

Was zum Software-Check gehört

Management-Summary und Sicherheitsdurchsicht wie in diesem Muster, Vorab-Stichpunkte, ein Abschlusstermin von etwa anderthalb Stunden mit Live-Nachweisen, das Papier zum Betrieb und bei Bedarf Fragen an eure Dienstleister. Zu jedem Befund Fundstelle, Einstufung und eine Empfehlung, die ihr so an eure KI geben könnt.

Der Preis richtet sich nach dem Umfang des Codes und steht nach dem kostenlosen Erstgespräch fest.

Erstgespräch: rocketbase.io/software-check · marten@rocketbase.io · 04173 2199520